Prompt Engineering Portfolio — ClinicalBridge

Project: ClinicalBridge: Bridging the Clinical Context Gap
Course: COP-3442 Prompt Engineering
University: Bahçeşehir University
Department: Artificial Intelligence Engineering Department
Professor: Binnur Kurt
Group Members: - Kenan Eliyan — 2286181 - Rama Tamimi — 2285460 - Joud Wardeh — 2282493

> **Educational prototype. Fictional data only. Not a clinical tool. No diagnoses. Clinician review always required.**

This portfolio documents the prompt-design work for all four agents: the v1 → v2 → v3 evolution, the before/after reasoning, the failure modes each change fixed, the few-shot exemplar strategy, the output-schema contract, and the safety guardrails. Full prompt files live in `prompts/<agent>/`; this document embeds the short versions and excerpts the long one.

Method: why three versions

Version	Theme	Typically fixes
v1	*Make it work* — plain role + task	vague, prose output, no schema, occasional diagnosis, no citations
v2	*Make it safe & structured* — JSON, safety rules, cite-or-omit	parseable; diagnosis suppressed; weak on uncertainty/conflicts
v3	*Make it robust* — few-shot, self-check, calibrated confidence, negative-space	handles edge cases, surfaces gaps/conflicts, traceability ≈ 1.0

Cross-agent technique catalogue

Technique	Agents	Purpose
Role / persona prompting	all	scope behavior, non-diagnostic stance
Rubric-in-prompt	triage	consistent urgency judgments
Cite-or-omit grounding	ehr, anamnesis, synthesis	kill hallucination; force traceability
Verbatim-copy	ehr	prevent dose/lab drift
Source-typing (`reported_by`)	anamnesis	don't over-trust self-report
Negative-space prompting	all (`data_gaps`)	"absence is information"
Few-shot exemplar	synthesis	lock tone + citation style
Verify-before-emit	synthesis	catch uncited claims / diagnosis language
Constrained framing vocabulary	synthesis	"factors to consider", never diagnoses

---

Agent 1 — Alert Triage Agent

Output schema summary (`schemas/triage_output.schema.json`): `urgency_level` (routine/monitor/urgent/emergent), `urgency_rationale` , `alert_validity` (likely_valid/possible_artifact/insufficient_data), `validity_rationale` , `retrieval_plan` (`ehr_focus` , `anamnesis_focus`), `missing_data_flags` , `safety_flags` , `confidence` , `citations` .

Safety guardrails: never diagnose (urgency ≠ diagnosis); when in doubt triage up; an artifact is flagged but still triggers retrieval; if it can't judge, output `insufficient_data` .

v1 (naive)

```
You are a triage assistant for a remote patient monitoring system. You will be given an
alert from a
patient's monitoring device. Look at the alert and tell me how serious it is and what might
be going
on. Then say what information would help.
```

v2 (structured + safety)

Adds a JSON contract, an urgency rubric, and the no-diagnosis rule (see `prompts/alert_triage_agent/v2.md`).

v3 (production, excerpt — full text in `prompts/alert_triage_agent/v3.md`)

```
Absolute rules
1. You do not diagnose. urgency_level is prioritization ONLY ... Never name a disease as the
```

```
cause.
2. When in doubt, triage UP, and say why.
3. An artifact is not dismissable ... STILL produce a retrieval plan.
4. If you cannot judge urgency ... output `insufficient_data` ... Do not guess.
...
Do not be anchored by the device's severity_raw — it is a crude device rating, not a
clinical judgment.
```

Before/after. v1 returned prose like "*This looks pretty serious, probably high blood pressure issues*" — unparseable and a soft diagnosis. v2 made it JSON and banned diagnosis. v3 added artifact / insufficient-data handling and the "ignore device severity" rule.

Failure-mode analysis. The key v2 failure was *anchoring to* `severity_raw`: a heart-failure weight trend rated "medium" by the device was under-triaged. v3's explicit instruction to ignore the raw severity makes the agent escalate Scenario 3 to `urgent` on the basis of the trend itself.

Agent 2 — EHR Retrieval Agent

Output schema summary (`schemas/ehr_retrieval_output.schema.json`): `relevant_diagnoses` , `relevant_medications` , `relevant_labs` , `relevant_allergies` , `relevant_visit_notes` (each item = `source_id` + verbatim `content` + `relevance_reason`), plus required `data_gaps` and `retrieval_confidence` .

Safety guardrails: copy values verbatim (no rounding/paraphrase); include only items present in the record (never infer); list relevant-but-absent data as `data_gaps` .

v1 (naive)

```
You are an assistant that reads a patient's electronic health record. Given an alert,
summarize the
patient's medical history that is relevant to the alert so a doctor can understand the
situation.
```

v2 (structured + grounding)

Adds JSON, mandatory `source_id` , per-item relevance reason (see `prompts/ehr_retrieval_agent/v2.md`).

v3 (production, excerpt)

```
1. Cite or omit. Every item ... MUST carry its real source_id ... never invent or infer ...
2. Copy values VERBATIM. Do not round, paraphrase, or "correct" a dose, a lab value, or a
date.
4. Negative space is required. In data_gaps, list relevant data that SHOULD exist ... but is
absent.
```

Before/after. v1 wrote a flowing narrative and once paraphrased a dose ("a moderate dose of lisinopril") with no sources. v2 forced source ids and verbatim copying. v3 added the required `data_gaps` array and per-metric relevance heuristics.

Failure-mode analysis. v2 *silently skipped missing data* — it never noted the absent recent potassium before an ACE-inhibitor BP alert, so the synthesis agent couldn't flag it. The negative-space requirement in v3 surfaces those gaps (Scenario 1 absent potassium; Scenario 3 stale potassium/creatinine).

Working with retrieved LangChain Documents (RAG). In the retrieval-augmented setup, the EHR Retrieval Agent no longer sees the whole chart as prose — it receives the **top-k retrieved LangChain `Document` objects** produced by a genuine LangChain RAG pipeline (`langchain_pipeline/`): EHR records → `Document`s → `RecursiveCharacterTextSplitter` → local `all-MiniLM-L6-v2` embeddings → **FAISS** → retriever (default `k=5`). The clinical question is built from the **Alert Triage Agent's** `retrieval_plan.ehr_focus` , the retriever returns the most relevant chunks (each `Document` carrying its `source_id` , `source_type` , and `patient_id` in `metadata`), and those Documents are attached to the EHR agent's context (`context["ehr_retrieval_documents"]`) as its grounding evidence. The agent's prompt then instructs it to use **only** those retrieved Documents and to **cite only their source ids** (cite-or-omit) — which is exactly what keeps retrieval grounded and the downstream brief traceable. This is a real RAG flow (retrieve → ground → cite) with **local embeddings and no API key**; if the LangChain stack is absent it degrades to the pure-Python TF-IDF retriever (`rag_retrieval.py`). Retrieval quality (Precision@k / Recall@k) is measured in `evaluation_report.md` .

Agent 3 — Anamnesis Agent

Output schema summary (`schemas/anamnesis_output.schema.json`): `relevant_symptoms` , `adherence_findings` , `lifestyle_factors` , `relevant_family_history` , `patient_concerns` (each item = `source_id` + `content` + `reported_by` + `relevance_reason`), `extraction_completeness` (present/absent/not_collected per domain), `data_gaps` .

Safety guardrails: everything tagged self-report; symptoms never converted to diagnoses; do not fabricate to fill a gap; preserve the patient's own words for concerns.

v1 (naive)

```
You are an assistant that reads the patient's interview notes. Pull out the important
symptoms and
history related to the alert and explain what they suggest.
```

v2 (structured + self-report typing)

Adds JSON, `reported_by` tags, source ids, "symptoms stay symptoms" (see `prompts/anamnesis_agent/v2.md`).

v3 (production, excerpt)

```
5. Handle absence explicitly ... do NOT fabricate to fill a gap. If the whole anamnesis was
not
   collected, return empty arrays and say so clearly.
Special case: anamnesis not collected -> all arrays empty, every completeness field
"not_collected",
   first data_gaps entry states clearly what cannot be assessed.
```

Before/after. v1 editorialized self-report into fact ("the patient is clearly developing heart failure") — both a diagnosis and a loss of provenance. v2 added self-report tagging and the no-diagnosis rule. v3 added explicit not-collected handling and the completeness map.

Failure-mode analysis. The critical v2 failure: in Scenario 4 the anamnesis was *not collected*, and v2 **hallucinated plausible symptoms and adherence** to fill the template — the single most dangerous behavior for this project. v3's not-collected block makes the agent return empty arrays and flag the gap, with zero fabrication. This is also the basis of adversarial Test C (vague symptom), where the agent preserves "*I feel a bit off*" verbatim and flags it non-specific.

Agent 4 — Synthesis Agent (most safety-critical)

Output schema summary (`schemas/clinical_context_brief.schema.json`): `alert_summary`, `patient_snapshot`, `contextual_findings`, `possible_contributing_factors` (factor + supporting_evidence + citations + uncertainty), `data_conflicts`, `missing_data_and_uncertainty`, `suggested_clinician_considerations`, `urgency_assessment`, `safety_disclaimer`, `traceability_index`, `overall_confidence`. The human-readable brief renders these as the 8 sections: Alert Summary, Patient Snapshot, Contextual Analysis, Risk Assessment, Recommended Actions, Uncertainties and Gaps, Sources Used, Safety Disclaimer.

Safety guardrails: no diagnosis ("factors to consider" only, banned-phrasing list); cite or omit; surface conflicts without resolving them; required uncertainty + disclaimer; considerations not orders; keep the higher urgency.

v1 (naive)

```
You are a clinical assistant. Given the triage result, the EHR information, and the patient
interview
findings, write a clear summary for the doctor explaining what is going on with the patient
and what
they should do.
```

v2 (structured + no-diagnosis + citations)

Adds JSON, "factors to consider" framing, per-claim citations (see `prompts/synthesis_agent/v2.md`).

v3 (production, excerpt — full text in `prompts/synthesis_agent/v3.md`)

```
2. Cite or omit. EVERY clinical claim ... MUST carry >=1 source_id ... Build a
traceability_index ...
3. Surface conflicts; never silently resolve them ... cite BOTH sources and do NOT pick a
winner.
4. Uncertainty and missing data are required output.
Verify-before-emit: walk every finding/factor/conflict — does each have a citation that
exists in the
   inputs and in traceability_index? ... Scan for any sentence that asserts a diagnosis ...
re-phrase
   or delete.
```

Few-shot exemplar strategy. The gold briefs in `scenarios/*.json` act as format exemplars: when generating a synthesis output manually, the gold brief for a similar scenario is shown to the model to lock the citation style ("[EHR.med.001]"), the hedged "factor to consider" voice, and the required sections. This is classic few-shot format conditioning rather than answer-leaking, because each scenario's data differs.

Few-shot example (abridged, from Scenario 1).

```
INPUT (abridged): triage{urgency:urgent}, ehr{lisinopril last refill 2026-05-02, 30-day
supply},
                  anamnesis{"ran out of my BP pill ~1 week ago"}
OUTPUT (abridged): possible_contributing_factors:[{
   factor:"Possible contribution from an antihypertensive medication lapse",
   citations:["ANAMNESIS.adherence.1","EHR.med.001","RPM.alert"],
   uncertainty:"Self-reported lapse not confirmed against pharmacy data ..." }]
```

Before/after. v1 produced "*The patient is in acute heart failure exacerbation; start IV diuresis.*" — a diagnosis **and** a treatment order. v2's "factors to consider" framing + cite-or-omit removed both and made claims traceable (safety FAIL → PASS).

Failure-mode analysis. v2's residual failure was on **conflicts**: in Scenario 5 it *silently chose* the EHR medication list over the patient's report, hiding the most important finding. v3's required `data_conflicts` + the verify-before-emit self-check force the contradiction to the surface with both sources cited, and drive the measured hallucination rate to 0.0 and traceability to 1.0.

Few-Shot Examples (≥3 per agent, including an edge case)

Per the brief (§8.1.2), each agent has at least three input → expected-output examples, including one edge case. These are **abridged** for readability; full versions live in the scenarios and `agent_outputs/`.

Alert Triage Agent

- **Example 1 (normal).** IN: `BP 182/104, limit 160/100, baseline 126/76`. OUT: `urgency=urgent`, `validity=likely_valid`, retrieval_plan asks for antihypertensives + renal labs + adherence.

- **Example 2 (normal).** IN: `HR 142 during device-logged cycling, baseline 68`. OUT: `urgency=routine`, `validity=likely_valid`, rationale = activity-explained; plan asks for cardiac history + caffeine.
- **Example 3 (edge — implausible reading).** IN: `SpO2 38% but patient conversant, no distress`. OUT: `validity=possible_artifact`, urgency conservative, **still** emits a retrieval_plan and a safety flag (do not silently dismiss). *(If a reading were both extreme and plausible, urgency would be `emergent` → official Critical → immediate escalation.)*

EHR Retrieval Agent

- **Example 1 (normal).** IN: BP alert + plan. OUT: returns `EHR.dx.001` (hypertension), `EHR.med.001` (lisinopril + refill date), renal labs — each verbatim with `source_id` + reason.
- **Example 2 (normal).** IN: glucose alert + plan. OUT: returns diabetes dx, insulin/metformin, HbA1c; leaves unrelated chart items out.
- **Example 3 (edge — sparse record).** IN: newly enrolled patient, labs pending. OUT: returns the few items that exist **and** a `data_gaps` entry ("renal/lipid labs pending; no baseline") — flags absence instead of inventing.

Anamnesis Agent

- **Example 1 (normal).** IN: BP-lapse interview. OUT: `adherence_findings` = "ran out of BP pill ~1 week ago", tagged `reported_by: patient`; no diagnosis.
- **Example 2 (normal).** IN: heart-failure check-in. OUT: symptoms "SOB on stairs", "ankles swollen"
- salty-diet lifestyle factor, all self-report.
- **Example 3 (edge — not collected).** IN: patient did not answer the call. OUT: all arrays empty, `extraction_completeness` all `not_collected`, first `data_gap` states the anamnesis was not collected — **zero fabrication**.

Synthesis Agent

- **Example 1 (normal).** IN: triage+EHR+anamnesis for the BP lapse. OUT: brief with a *factor to consider* (medication lapse) cited to `ANAMNESIS.adherence.1` + `EHR.med.001`, missing-data flag, disclaimer.
- **Example 2 (normal).** IN: the heart-failure trend. OUT: trend-based factor cited across weight readings + symptoms; urgency kept at the higher level; uncertainty noted.
- **Example 3 (edge — conflicting sources).** IN: EHR lists metformin active, patient says stopped. OUT: a `data_conflicts` entry citing **both** sources, explicitly **not** resolved, routed to medication reconciliation.

Urgency Taxonomy Mapping (official COP-3442 brief)

The official brief uses **Critical / Urgent / Routine / Informational**. The project's internal schema enum is kept unchanged (to keep all stored outputs, gold standards, and evaluation valid); the rendered brief shows the official label, mapped as:

Internal (schema enum)	Official (brief taxonomy)
`emergent`	**Critical**
`urgent`	**Urgent**
`routine`	**Routine**
`monitor`	**Informational**

The mapping is implemented in `pipeline.official_urgency()` and shown in §4 (Risk Assessment) of every rendered brief.

Model Parameter Rationale

- **No-API / manual mode stores Claude outputs for reproducibility.** Because outputs are generated once and replayed, the demo, brief, and evaluation are byte-identical on every run — ideal for grading.
- **For a live Claude API later, prefer a low temperature (≈ 0.0–0.2)** for these structured clinical-context tasks: the goal is reliable, schema-valid JSON with calibrated language, not creative variation.
- `max_tokens` **should be high enough to fit the full JSON brief but constrained** to discourage rambling or fabricated padding.
- **Deterministic settings support evaluation repeatability** — the same input should yield the same structured output so metric deltas reflect prompt changes, not sampling noise.
- **Safety-critical domains prioritize consistency over creativity**; low-temperature, schema-enforced decoding is the appropriate default here.

Summary of measured impact

Dimension	v1	v3
Output parseable as JSON	no	yes (schema-valid)
Diagnosis language	present (unsafe)	absent (safety PASS)
Treatment orders	present (unsafe)	absent (considerations only)
Claims cited	rarely	every claim (traceability 1.0)
Hallucination rate	high/unmeasured	0.0 on tested scenarios
Conflicts surfaced	no	yes (Scenario 5)
Fabrication on empty input	yes (Scenario 4)	no (flagged not-collected)

> All examples use fictional data and illustrate prompt behavior only; nothing here is medical advice.